

Docker Containerisation Guide

Thu, Aug 20 at 11:08 PM



Tell me everything about containerisation , how it works , why it is needed , when it is needed. How docker works , solved the problem how many more containers available like docker , why docker is good , list every important commands needed and when to use it , give me each and every details to remember of containers images process docker

Absolutely. Since you're currently learning Docker + Linux, the best way to understand Docker is **not to memorize commands first**. First understand the problem containers solve, then understand what a container actually is, then Docker's architecture, and only after that learn the commands.

Think of this as your **Docker + Containerisation master note**.

1. What problem existed before containers?

Suppose you build a backend application.

Your application needs:

- Python 3.12
- Django 5.x
- PostgreSQL
- Redis
- specific system libraries
- environment variables
- certain versions of packages

It works perfectly on your laptop.

Then you give it to another developer.

They install it and get:

"Works on my machine."

Because their machine has:

- Python 3.11
- different PostgreSQL
- different libraries
- different OS configuration
- different dependency versions

This is the fundamental problem:

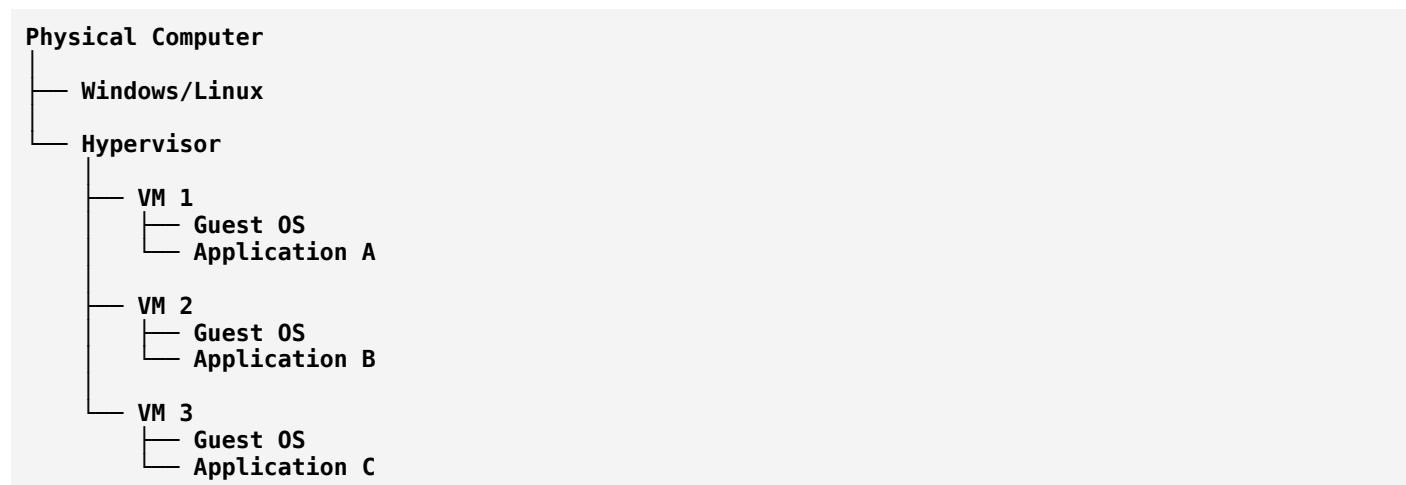
An application doesn't just depend on its source code. It depends on its entire runtime environment.

2. Before containers: Virtual Machines

One solution was:

Give every application its own Virtual Machine.

For example:



This provides excellent isolation.

But there is a problem.

Each VM has its **own operating system**.

If you have:

```
VM 1 → Ubuntu
VM 2 → Ubuntu
VM 3 → Ubuntu
VM 4 → Ubuntu
```

you're running four operating systems.

That's expensive in:

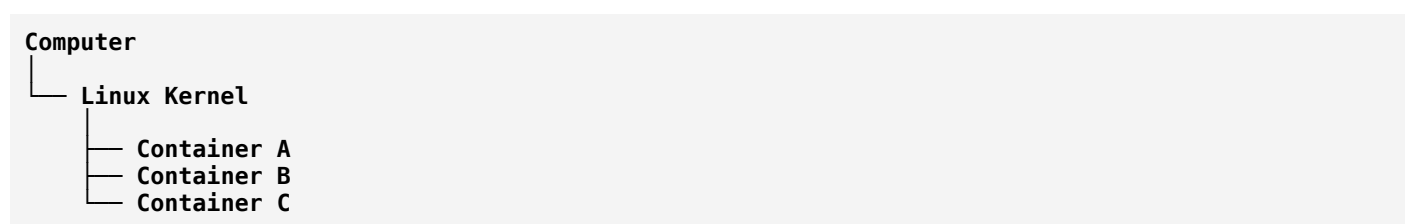
- RAM
- CPU
- disk
- startup time

A VM might take seconds/minutes to start and consume hundreds of MBs or GBs.

3. Then came containers

Containers take a different approach.

Instead of giving every application an entire OS:



All containers share the **host kernel**.

But they appear to have their own isolated environments.

That's the key idea.

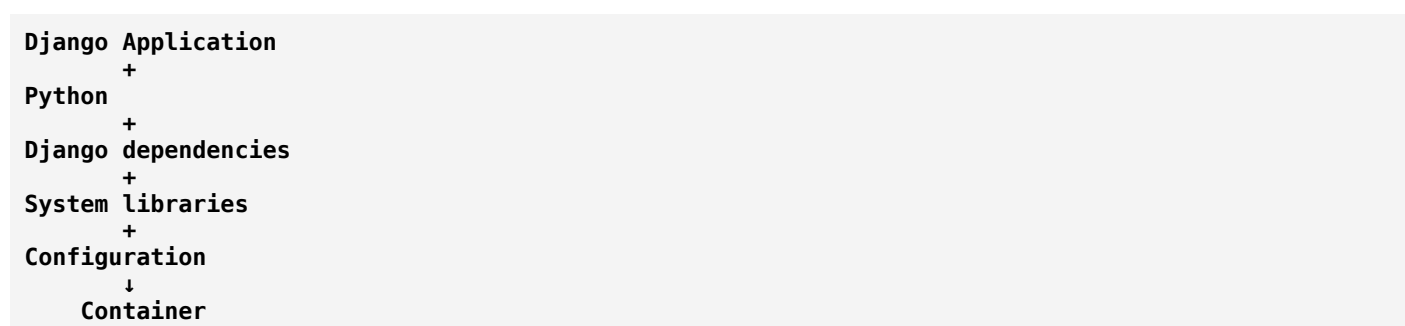
4. What is containerisation?

Containerisation is a method of packaging an application together with:

- application code
- dependencies
- libraries
- configuration
- runtime environment

into an isolated unit called a **container**.

For example:



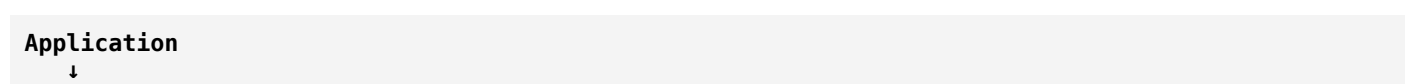
You can then run that same container on another compatible machine.

5. The most important misconception

A container is **NOT a lightweight virtual machine**.

This is extremely important.

A VM:



```

Guest OS
↓
Virtual Hardware
↓
Hypervisor
↓
Host OS
↓
Hardware

```

Container:

```

Application
↓
Container filesystem/environment
↓
Host Linux Kernel
↓
Hardware

```

Containers don't normally contain their own kernel.

6. Then what is Ubuntu inside Docker?

This confused you recently.

You ran:

```

Bash
docker run -it ubuntu bash

```

and Docker downloaded an Ubuntu image.

You might think:

"Docker installed Ubuntu."

Not exactly.

The Ubuntu Docker image contains a **user-space filesystem** resembling Ubuntu:

```

/bin
/etc
/home
/lib
/usr
/var
...

```

But it doesn't contain a complete Ubuntu kernel.

The kernel comes from the underlying Linux environment.

That's why:

Docker containers are fundamentally Linux processes isolated by Linux kernel features.

7. The three words you MUST understand

If you remember nothing else initially, remember:

```
IMAGE → CONTAINER → PROCESS
```

And also:

```
IMAGE = blueprint  
CONTAINER = running instance  
PROCESS = actual program running inside it
```

For example:

```
ubuntu image  
  ↓  
docker run  
  ↓  
ubuntu container  
  ↓  
bash process
```

8. What is an image?

An image is a **read-only template** used to create containers.

For example:

```
ubuntu:24.04
```

is an Ubuntu image.

Another:

```
postgres:18
```

is a PostgreSQL image.

Another:

```
python:3.12
```

is a Python image.

Another:

```
nginx:latest
```

is an Nginx image.

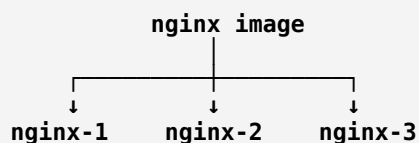
Think:

```
Class → Object
```

Similarly:

```
Image → Container
```

One image can create many containers.



9. What is a container?

A container is a **running or stopped instance created from an image**.

For example:

```
Bash
docker run nginx
```

Docker:

```
nginx image
  ↓
creates container
  ↓
starts nginx process
```

You can have:

```
nginx image
  ↓   ↓   ↓
container1 container2 container3
```

All three can be created from the same image.

10. What is a process?

This is another important distinction.

Suppose:

```
Bash
docker run ubuntu sleep 1000
```

Inside the container, the actual running program is:

```
sleep
```

A container isn't magic.

Underneath, the host kernel sees processes.

For example:

```
Host Linux
├── PID 100 → Docker daemon
├── PID 500 → nginx
├── PID 501 → postgres
└── PID 502 → python
```

Containers isolate these processes so they appear to have their own environment.

11. How does Linux actually make containers possible?

Docker didn't invent containers from scratch.

The Linux kernel provides mechanisms such as:

Namespaces

Used for isolation.

cgroups

Used for resource control.

capabilities

Used to control privileges.

seccomp

Used to restrict system calls.

Linux filesystem features

Used for container filesystems.

These are the fundamental technologies behind containerisation.

12. Namespaces

Namespaces make a process think it has its own world.

Important namespaces include:

PID namespace

Controls process visibility.

Container might see:

```
PID 1 → nginx
PID 2 → worker
```

while the host might see:

```
PID 15432 → nginx
PID 15433 → worker
```

Inside the container:

```
Bash
ps
```

may show only container processes.

Network namespace

Provides an isolated network environment.

A container can have:

```
eth0
IP address
routing table
ports
```

separate from another container.

Mount namespace

Controls what filesystem mounts the process sees.

This contributes to the container having its own filesystem view.

UTS namespace

Allows isolation of:

- hostname
- domain name

IPC namespace

Isolates inter-process communication resources.

User namespace

Can map users inside the container to different users on the host.

13. cgroups

Namespaces answer:

"What can the process see?"

cgroups answer:

"How many resources can the process use?"

For example:

```
Container A
CPU → 1 core
RAM → 512 MB

Container B
CPU → 2 cores
RAM → 2 GB
```

You can restrict:

- CPU
- memory
- processes
- I/O

etc.

Without resource limits, one badly behaved container could consume huge amounts of resources.

14. Container filesystem

Containers need a filesystem.

Docker commonly uses layered filesystems.

Imagine an image:

```
Layer 4 → Application
Layer 3 → Python packages
Layer 2 → Python
Layer 1 → Debian/Ubuntu base
```

These layers are generally read-only.

When the container starts, Docker adds a writable layer on top.

Conceptually:

```
Application layer
Python packages layer
Python layer
Base OS layer
-----
Writable container layer
```

This is why Docker images can be reused efficiently.

15. Why layers?

Suppose you have:

```
Image A
Ubuntu
Python
Django
App A
```

```
Image B
Ubuntu
Python
Django
App B
```

You don't want to duplicate:

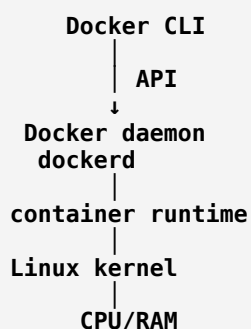
```
Ubuntu
Python
Django
```

Docker can reuse common image layers.

That's one reason images are efficient.

16. Docker's architecture

Docker typically has:



You interact with Docker using:

```
Bash
docker
```

The Docker CLI communicates with the Docker daemon.

17. Docker CLI

When you run:

```
Bash
docker run nginx
```

the CLI sends a request to the Docker engine/daemon.

Conceptually:

```

You
↓
docker run nginx
↓
Docker CLI
↓
Docker daemon
↓
Find nginx image
↓
Pull image if necessary
↓
Create container
↓
Configure filesystem/network
↓
Start process

```

18. Docker daemon

The Docker daemon is commonly:

```
dockerd
```

It manages things such as:

- images
- containers
- networks
- volumes

You don't normally interact with it directly.

You use:

```

Bash
docker ...

```

19. Docker container runtime

Docker uses lower-level container runtime components.

Modern Docker uses components such as:

```

Docker
↓
containerd
↓
runc
↓
Linux kernel

```

Simplified:

```
Docker CLI
  ↓
Docker Engine
  ↓
containerd
  ↓
runc
  ↓
Linux kernel
```

You don't need to memorize every internal component immediately.

But knowing the hierarchy helps:

Docker is not itself the Linux kernel container mechanism.

Docker orchestrates the machinery.

20. Docker Hub

Where do images come from?

Usually from a registry.

The most famous is:

Docker

and its public registry, Docker Hub.

Conceptually:

```
Docker Hub
  |
  | docker pull
  ↓
Your computer
  |
  ↓
Docker image
  |
  ↓
Container
```

For example:

```
Bash
docker pull nginx
```

Docker downloads the image.

21. Registry vs repository vs image

These terms are easy to mix up.

Registry

A place that stores images.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Examples:

```
Docker Hub
GitHub Container Registry
Amazon ECR
Google Artifact Registry
Azure Container Registry
Harbor
```

Repository

A collection/name for an image.

Example:

```
nginx
```

Tag

A version/variant identifier.

```
nginx:1.27
nginx:latest
```

So:

```
registry / repository : tag
```

Example:

```
docker.io/library/nginx:latest
```

22. Image naming

When you write:

```
Bash
docker pull nginx
```

Docker interprets it roughly as:

```
docker.io/library/nginx:latest
```

Where:

```
docker.io = registry
library/nginx = repository
latest = tag
```

23. latest does NOT mean newest

Very important.

```
Bash
nginx:latest
```

doesn't mathematically mean:

"the newest version available."

It means:

"the image tagged **latest**."

Tags can be moved.

For production, version pinning is usually safer.

For example:

```
postgres:18
```

or even a digest when strict reproducibility is required.

24. The most important Docker command

```
Bash
docker run
```

Example:

```
Bash
docker run nginx
```

This essentially means:

Create a container from the nginx image and start it.

If image isn't locally available:

```
docker run
  ↓
image exists?
  ↓
NO
  ↓
pull image
  ↓
create container
  ↓
start container
```

25. docker pull

Downloads an image.

```
Bash
```

```
docker pull ubuntu
```

Specific tag:

```
Bash
docker pull ubuntu:24.04
```

Use when:

- you want to download an image
- you don't want to start a container yet

26. docker images

Lists local images.

```
Bash
docker images
```

or:

```
Bash
docker image ls
```

You'll see things like:

REPOSITORY	TAG	IMAGE ID
ubuntu	24.04	...
nginx	latest	...
postgres	18	...

27. docker ps

Shows running containers.

```
Bash
docker ps
```

Example:

```
CONTAINER ID
IMAGE
COMMAND
STATUS
PORTS
NAMES
```

28. docker ps -a

Shows:

running + stopped containers.

This is extremely important.

```
Bash
docker ps
```

means:

"What containers are currently running?"

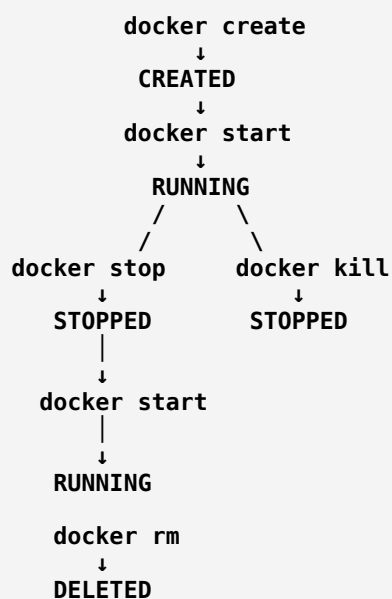
```
Bash
docker ps -a
```

means:

"What containers exist?"

29. Container lifecycle

Understand this diagram:



And:

```
Bash
docker restart
```

means:

```
stop + start
```

roughly speaking.

30. docker create

Creates a container but doesn't start it.

```
Bash
docker create nginx
```

Then:

```
Bash
docker start <container>
```

Most of the time you'll use:

```
Bash
docker run
```

because it creates + starts.

31. docker start

Starts an existing stopped container.

```
Bash
docker start mycontainer
```

32. docker stop

Gracefully stops a container.

```
Bash
docker stop mycontainer
```

This sends a termination signal and gives the process time to shut down.

Prefer this over immediately killing applications.

33. docker kill

Forcefully terminates the container's main process.

```
Bash
docker kill mycontainer
```

Use when:

- process isn't responding

- you deliberately need immediate termination

34. docker restart

```
Bash
docker restart mycontainer
```

Useful for restarting services.

35. docker rm

Deletes a stopped container.

```
Bash
docker rm mycontainer
```

Important:

Removing a container does NOT necessarily remove its image.

36. docker rmi

Removes an image.

```
Bash
docker rmi nginx
```

Modern equivalent:

```
Bash
docker image rm nginx
```

You generally can't remove an image if containers still depend on it unless you remove/force things appropriately.

37. Container names

Docker automatically assigns names such as:

```
happy_turing
```

But you can specify one:

```
Bash
docker run --name my-nginx nginx
```

Now:

```
Bash
docker ps
```

shows:

```
my-nginx
```

This is much easier to work with.

38. Interactive containers

You previously used:

```
Bash
docker run -it ubuntu bash
```

Understand the flags:

```
-i → interactive
-t → pseudo-terminal
```

So:

```
Bash
docker run -it ubuntu bash
```

means approximately:

Start Ubuntu container and attach my terminal to a bash process.

39. docker exec

This is one of the most important commands.

Suppose:

```
container is already running
```

You want to enter it.

Use:

```
Bash
docker exec -it mycontainer bash
```

or:

```
Bash
```

```
docker exec -it mycontainer sh
```

Difference:

```
docker run
```

creates a new container.

```
docker exec
```

runs another process inside an existing running container.

This distinction is **very important**.

40. docker attach

```
Bash
docker attach mycontainer
```

Attaches your terminal to the container's main process.

Don't confuse:

```
Bash
docker exec
```

with:

```
Bash
docker attach
```

Usually, when debugging a running container, **exec** is safer and more useful.

41. Container logs

If an application prints logs to stdout/stderr:

```
Bash
docker logs mycontainer
```

Follow logs:

```
Bash
docker logs -f mycontainer
```

Very important for debugging.

For example:

```
Bash
```

```
docker logs postgres
```

might tell you why PostgreSQL failed to start.

42. docker inspect

Extremely useful.

```
Bash
docker inspect mycontainer
```

Shows detailed metadata:

- networking
- mounts
- environment
- image
- command
- state
- IDs
- configuration

When you don't understand what's happening:

```
Bash
docker inspect
```

is your friend.

43. docker stats

Shows resource usage:

```
Bash
docker stats
```

You'll see:

```
CPU
Memory
Network
Block I/O
PIDs
```

Useful for monitoring containers.

44. Port mapping

This is one of the most important Docker concepts.

Suppose nginx inside the container listens on:

```
port 80
```

But the container has its own network.

Your host can't necessarily access it directly through:

```
localhost:80
```

You can publish it:

```
Bash
docker run -p 8080:80 nginx
```

Meaning:

```
HOST PORT : CONTAINER PORT
```

So:

```
localhost:8080
  ↓
Docker
  ↓
container:80
  ↓
nginx
```

45. EXPOSE vs -p

This causes confusion.

Dockerfile:

```
dockerfile
EXPOSE 80
```

does **not** publish the port to your host.

It mainly documents/declares the intended container port.

Actual publishing:

```
Bash
docker run -p 8080:80 nginx
```

46. Docker networking

Containers can communicate over Docker networks.

For example:



You can create a network:

```
Bash
docker network create mynetwork
```

Run containers on it:

```
Bash
docker run --network mynetwork --name backend my-backend
```

```
Bash
docker run --network mynetwork --name db postgres
```

Then the backend can often connect to:

```
db
```

rather than hardcoding an IP address.

47. Why container names matter

Suppose:

```
backend
database
redis
```

All belong to the same Docker network.

Then applications can communicate using names:

```
database:5432
redis:6379
```

Docker's network DNS handles name resolution.

That's much better than:

```
192.168.1.27
```

because container IP addresses can change.

48. Common network commands

Create:

```
Bash
docker network create mynetwork
```

List:

```
Bash
docker network ls
```

Inspect:

```
Bash
docker network inspect mynetwork
```

Remove:

```
Bash
docker network rm mynetwork
```

Connect:

```
Bash
docker network connect mynetwork container
```

Disconnect:

```
Bash
docker network disconnect mynetwork container
```

49. Volumes

This is **critical**.

Containers are disposable.

Suppose PostgreSQL stores data inside the container:

```
Postgres container
↓
database files
```

You delete the container:

```
Bash
docker rm postgres
```

Your data may disappear with it.

That's why persistent data should live outside the container's writable layer.

Enter:

Volumes

50. Docker volumes

Create:

```
Bash
docker volume create pgdata
```

List:

```
Bash
docker volume ls
```

Inspect:

```
Bash
docker volume inspect pgdata
```

Use:

```
Bash
docker run \
  --name postgres \
  -v pgdata:/var/lib/postgresql/data \
  postgres
```

Conceptually:

```
Postgres container
  |
  | /var/lib/postgresql/data
  |
  v
Docker volume
  |
  v
Persistent storage
```

Now deleting the container doesn't automatically delete the volume.

51. Bind mounts

You can mount a host directory directly.

Example:

```
Bash
docker run -v ./app:/app myimage
```

Conceptually:

```
Host
./app
|
| bind mount
|
↓
Container
/app
```

This is extremely useful during development.

52. Volume vs bind mount

Volume

Managed by Docker:

```
Bash
→ myvolume:/data
```

Good for:

- databases
- persistent application data

Bind mount

You choose the exact host path:

```
Bash
→ ./src:/app/src
```

Good for:

- development
- source code
- live editing

53. Environment variables

Containers commonly receive configuration through environment variables.

```
Bash
docker run \
-e POSTGRES_PASSWORD=secret \
postgres
```

Inside container:

```
Bash
```

```
echo $POSTGRES_PASSWORD
```

You can inspect environment configuration with:

```
Bash
docker inspect
```

But don't casually put production secrets directly into commands or Dockerfiles.

54. Dockerfile

Now we move from:

"How do I run existing images?"

to:

"How do I create my own image?"

That's what a Dockerfile is for.

Example:

```
dockerfile
FROM python:3.12
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

This is essentially a recipe for building an image.

55. Dockerfile instructions you should know

FROM

Base image.

```
dockerfile
FROM python:3.12
```

WORKDIR

Sets working directory.

```
dockerfile
```

WORKDIR /app

COPY

Copies files into image.

```
dockerfile
COPY . .
```

RUN

Executes command during image build.

```
dockerfile
RUN pip install -r requirements.txt
```

CMD

Default command when container starts.

```
dockerfile
CMD ["python", "app.py"]
```

ENTRYPOINT

Defines the main executable behavior.

```
dockerfile
ENTRYPOINT ["python"]
```

ENV

Sets environment variables.

```
dockerfile
ENV PORT=8000
```

EXPOSE

Documents intended container port.

```
dockerfile
EXPOSE 8000
```

USER

Choose user under which application runs.

```
dockerfile
```

USER appuser

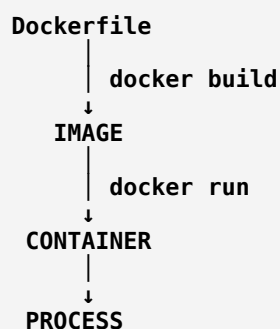
ARG

Build-time variable.

```
dockerfile
ARG VERSION=1
```

56. Dockerfile → Image → Container

This is another diagram you should memorize:



57. Build an image

Suppose current directory contains:

```
Dockerfile
app.py
requirements.txt
```

Run:

```
Bash
docker build -t myapp .
```

Meaning:

```

docker build
  ↓
-t myapp      → give image a name/tag
.             → build context = current directory
  
```

58. Build context

The . in:

```
Bash
docker build -t myapp .
```

is important.

It means:

Send the current directory as the build context.

Docker can only access files inside the build context when executing **COPY/ADD**.

That's why **.dockerignore** is important.

59. .dockerignore

Similar concept to **.gitignore**.

Example:

```
node_modules
.git
.env
__pycache__
*.log
```

Why?

You don't want to send:

```
node_modules/
.git/
secrets
large files
```

to the Docker build context unnecessarily.

60. Docker image layers

Each Dockerfile instruction can contribute to image layers.

For example:

```
dockerfile
FROM python:3.12
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Docker can cache previous build steps.

This is why Dockerfile ordering matters.

A common optimization:

```
dockerfile
```

```
COPY requirements.txt .  
RUN pip install -r requirements.txt  
  
COPY . .
```

Instead of:

```
dockerfile  
COPY . .  
RUN pip install -r requirements.txt
```

Because application code changes frequently, while dependencies may not.

61. Docker Compose

When applications become realistic, you rarely have only one container.

Imagine:

```
Frontend  
Backend  
PostgreSQL  
Redis  
Nginx
```

Running all of these manually is annoying.

That's where Docker Compose comes in.

You define services in a YAML file.

Conceptually:

```
compose.yaml  
  
services:  
  backend  
  postgres  
  redis  
  frontend
```

Then:

```
Bash  
docker compose up
```

starts the application stack.

62. Docker Compose commands

Start:

```
Bash  
docker compose up
```

Background:

```
Bash
docker compose up -d
```

Stop/remove stack:

```
Bash
docker compose down
```

Build:

```
Bash
docker compose build
```

Build and start:

```
Bash
docker compose up --build
```

View services:

```
Bash
docker compose ps
```

Logs:

```
Bash
docker compose logs
```

Follow logs:

```
Bash
docker compose logs -f
```

Execute command:

```
Bash
docker compose exec backend bash
```

63. What problem Docker solved

Docker/containerisation primarily solved:

1. Environment consistency

```
Developer machine
Testing server
Production server
```


can run the same image.

2. Dependency isolation

Application A:

```
Python 3.10
```

Application B:

```
Python 3.12
```

can coexist.

3. Deployment consistency

Instead of:

"Install Python, then install these packages, then configure this..."

you can deploy:

```
image
```

4. Fast startup

Containers generally start much faster than VMs because they don't need to boot a full guest OS.

5. Efficient resource usage

Many containers can run on one host.

6. Scalability

You can run:

```
backend-1  
backend-2  
backend-3  
backend-4
```

from the same image.

7. Reproducibility

The same image can be tested and deployed.

64. What Docker did NOT solve

Docker isn't magic.

It does **not automatically solve**:

- bad application architecture
- security
- database design
- distributed systems
- monitoring
- backups
- scaling strategy
- orchestration
- networking complexity
- application bugs

Docker is a tool.

Don't turn it into a religion. 😊

65. Containers are ephemeral

This word is important.

Ephemeral means temporary.

A container should generally be treated as disposable.

For example:

```
Old backend container
  ↓
destroy
New backend container
  ↓
created from same image
```

Your application should survive this.

That's why:

```
code → image
data → volume/database
configuration → environment/secrets
```

66. The golden container principle

A good mental model is:

```

Container
↓
Run one primary service/process
↓
Keep it disposable
↓
Keep persistent data outside

```

For example:

```

GOOD

Postgres container
Redis container
Backend container
Nginx container

```

rather than:

```

One giant container
├── nginx
├── postgres
├── redis
├── backend
└── everything else

```

There are exceptions, but separation is the common pattern.

67. Docker vs VM

Feature	VM	Container
Isolation	Strong	Strong, but different mechanism
Guest OS	Yes	No separate kernel
Startup	Slower	Usually much faster
Resource usage	Higher	Lower
Kernel	Separate guest kernel	Shared host kernel
Packaging	VM image	Container image
Typical use	Full OS isolation	Applications/services

68. Docker isn't the only container technology

There are several alternatives.

Important ones:

Podman

Very important Docker alternative.

```

Docker
Podman

```

Podman is particularly popular in Linux/server environments and has a Docker-compatible CLI style.

containerd

A major container runtime/project.

Docker itself uses containerd internally.

CRI-O

Container runtime focused heavily on Kubernetes environments.

LXC / LXD

Linux container technologies.

They have a somewhat different philosophy and can be more system-container oriented.

Kubernetes

Important distinction:

Kubernetes is NOT a Docker replacement.

Kubernetes is a **container orchestration platform**.

It manages containers across machines.

Think:

```
Docker/Podman
  ↓
run containers
```

while:

```
Kubernetes
  ↓
manage huge numbers of containers
  ↓
across many machines
```

Modern Kubernetes commonly uses runtimes such as containerd or CRI-O.

69. Docker vs Podman

Roughly:

```
Docker
├── Docker CLI
├── Docker daemon
└── container runtime stack
```

```
Podman
├── Podman CLI
└── daemonless architecture
```

Podman can often use commands very similar to Docker.

For example:

```
Bash
podman run nginx
```

instead of:

```
Bash
docker run nginx
```

70. Docker vs Kubernetes

This distinction will save you a lot of confusion.

Docker

Good for:

```
build image
run container
network containers
store volumes
development
local environments
```

Kubernetes

Good for:

```
deploy many containers
scale
service discovery
load balancing
rolling updates
self-healing
cluster management
```

You usually learn:

```
Linux
↓
Docker/container fundamentals
↓
Docker Compose
↓
Kubernetes
```

That progression makes much more sense than jumping straight into Kubernetes YAML hell.

71. Important Docker commands — cheat sheet

Images

Bash

```
docker images
docker image ls
docker pull IMAGE
docker build -t NAME .
docker image inspect IMAGE
docker image rm IMAGE
docker tag SOURCE TARGET
docker history IMAGE
```

Use:

Bash

```
docker history image
```

to understand image layers.

72. Containers

Bash

```
docker ps
docker ps -a
docker run IMAGE
docker create IMAGE
docker start CONTAINER
docker stop CONTAINER
docker restart CONTAINER
docker kill CONTAINER
docker rm CONTAINER
```

73. Container debugging

Bash

```
docker logs CONTAINER
docker logs -f CONTAINER
docker exec -it CONTAINER bash
docker inspect CONTAINER
docker stats
docker top CONTAINER
```

These are some of the most useful commands you'll use professionally.

74. Networking

Bash

```
docker network ls
docker network create NAME
docker network inspect NAME
docker network connect NAME CONTAINER
docker network disconnect NAME CONTAINER
docker network rm NAME
```

75. Volumes

Bash

```
docker volume ls
docker volume create NAME
docker volume inspect NAME
docker volume rm NAME
```

76. Cleanup commands

Be careful with these.

Remove stopped containers:

Bash

```
docker container prune
```

Remove unused images:

Bash

```
docker image prune
```

Remove unused networks:

Bash

```
docker network prune
```

Remove unused volumes:

Bash

```
docker volume prune
```

Remove unused Docker objects:

Bash

```
docker system prune
```

More aggressive:

Bash

```
docker system prune -a
```

Don't blindly run prune commands on a machine containing important data.

77. docker system df

Very useful.

Bash

```
docker system df
```

Shows Docker disk usage.

If Docker is eating 50 GB of your SSD, this helps you figure out why.

78. Detached mode

Normally:

```
Bash
docker run nginx
```

may attach to the process depending on the container.

For background execution:

```
Bash
docker run -d nginx
```

-d means:

detached mode.

Then:

```
Bash
docker ps
```

to see it running.

79. Automatically remove container

Useful for temporary containers:

```
Bash
docker run --rm ubuntu echo hello
```

After the process finishes:

```
container → automatically removed
```

Very useful for testing.

80. Naming and ports together

A very common command:


```
Bash
docker run -d \
  --name web \
  -p 8080:80 \
  nginx
```

Read it as:

```
-d
→ run in background

--name web
→ container name = web

-p 8080:80
→ host 8080 → container 80

nginx
→ image
```

81. Resource limits

You can limit memory:

```
Bash
docker run --memory=512m nginx
```

CPU:

```
Bash
docker run --cpus=1 nginx
```

These are useful when you need predictable resource consumption.

82. Docker's storage hierarchy

Understand these three:

```
Image
↓
Container writable layer
↓
Volume / bind mount
```

Image:

```
read-only template
```

Container writable layer:

```
temporary
```

Volume:

persistent

83. Why database containers need volumes

Suppose:

```
postgres container
↓
database files
```

You remove:

```
Bash
docker rm postgres
```

The container filesystem goes away.

But:

```
postgres volume
```

can remain.

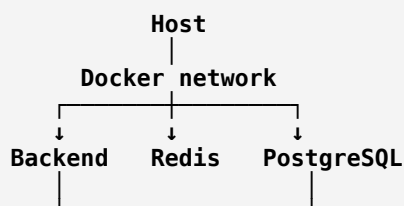
Then:

```
new postgres container
↓
same volume
↓
old database data
```

This is the proper pattern.

84. Container networking mental model

Imagine:



Backend can communicate with:

```
postgres:5432
redis:6379
```

rather than using host networking hacks.

85. Container lifecycle vs image lifecycle

Images and containers have separate lifecycles.

Image

```
pull
↓
exists
↓
remove
```

Container

```
create
↓
start
↓
running
↓
stop
↓
start again
↓
remove
```

One image can produce many containers.

86. Container ID vs Image ID

Container:

CONTAINER ID

identifies a particular container.

Image:

IMAGE ID

identifies an image.

Don't confuse:

image ≠ container

87. Image vs container: memorize this

Imagine you have a cake recipe.

```
Recipe = Image
Cake = Container
Eating/serving = Process/application execution
```

More technically:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

```
Dockerfile
  ↓ build
Image
  ↓ run
Container
  ↓ start
Process
```

That's the hierarchy you should keep in your head.

88. Docker networking modes you should know

Docker has several network modes.

bridge

Common default for containers.

```
container
  ↓
Docker bridge
  ↓
host network
```

host

Container shares the host's network namespace.

```
Bash
docker run --network host ...
```

This reduces network isolation and behaves differently from normal bridge networking.

none

No normal network connectivity:

```
Bash
docker run --network none ...
```

Useful for certain isolated workloads.

89. Docker volumes vs container filesystem

This is worth repeating.

Don't think:

"Container is a little computer where I permanently install things."

Think:

"Container is a disposable environment for running an application."

Persistent state should normally live somewhere designed for persistence.

90. What happens when you run `docker run nginx`?

Let's follow the entire journey.

You type:

```
Bash
docker run nginx
```

Step 1

Docker CLI receives the command.

Step 2

Docker daemon receives the request.

Step 3

Docker checks:

```
Do I have nginx image?
```

Step 4

If not:

```
Docker Registry
  ↓
download image
```

Step 5

Docker stores image layers locally.

Step 6

Docker creates a container from the image.

Step 7

Docker creates/configures:

- filesystem
- network
- namespaces

- cgroups/resource settings
- mounts

Step 8

Container runtime starts the container's process.

Step 9

Nginx starts.

Conceptually:

```

docker run nginx
  ↓
CLI
  ↓
Docker daemon
  ↓
Image lookup
  ↓
Pull if necessary
  ↓
Create container
  ↓
Namespaces
  ↓
cgroups
  ↓
Filesystem
  ↓
Networking
  ↓
container runtime
  ↓
nginx process

```

That is **Docker in action**.

91. Why does a container stop when its main process exits?

This is fundamental.

Suppose:

```

Bash
docker run ubuntu echo hello

```

The **echo** process executes:

```

hello

```

Then exits.

The container stops.

Why?

Because a container is fundamentally designed around its process lifecycle.

If the main process exits:

```
PID 1 exits
↓
container stops
```

This is why:

```
Bash
docker run ubuntu bash
```

may stop when bash exits.

92. PID 1 inside a container

The main process inside a container generally becomes PID 1 **inside that container's PID namespace**.

For example:

```
Container
PID 1 → nginx
PID 2 → worker
PID 3 → worker
```

PID 1 has special signal/reaping responsibilities.

This becomes increasingly important when you learn production container behavior.

93. Why one main process?

This is a useful Docker design principle:

```
One container
↓
One primary responsibility
```

Examples:

```
nginx container → nginx
postgres container → postgres
redis container → redis
backend container → backend
```

Then Compose/Kubernetes can combine them.

94. Docker image registries besides Docker Hub

Important ones to know:

- Docker Hub
- GitHub Container Registry

- Amazon Elastic Container Registry (ECR)
- Google Artifact Registry
- Azure Container Registry
- GitLab Container Registry
- Harbor
- Quay

So production architecture can look like:

```

Developer
↓
docker build
↓
Docker image
↓
Private registry
↓
Production server
↓
docker pull
↓
container

```

95. CI/CD + Docker

Docker becomes extremely powerful with CI/CD.

For example:

```

Developer
↓
git push
↓
GitHub Actions
↓
run tests
↓
docker build
↓
docker push
↓
Container Registry
↓
Production
↓
docker pull
↓
new container

```

This is one of the major reasons Docker is so popular professionally.

96. Docker Compose vs Dockerfile

Don't confuse them.

Dockerfile

Defines:

How to build ONE image.

```
Dockerfile
↓
image
```

Compose

Defines:

How multiple containers/services work together.

```
compose.yaml
↓
backend
database
redis
frontend
```

So:

```
Dockerfile → image
Compose → application stack
```

97. Docker vs container

Another important distinction:

Docker is a platform/toolset for working with containers.

Containerisation is the broader technology/concept.

There are container technologies besides Docker.

Docker is simply one of the most popular ecosystems.

98. Why Docker became so popular

Docker made containerisation accessible to ordinary developers.

Before Docker, working directly with Linux container technologies could be much more cumbersome.

Docker gave developers:

```
Dockerfile
docker build
docker run
docker push
docker pull
Docker Hub
Docker Compose
```

This made container workflows much easier.

The killer idea was essentially:

Build once, run consistently across environments.

Not perfectly identically in every possible situation, but dramatically more reproducibly.

99. Docker's limitations

Don't fall into the trap of thinking:

Docker = production architecture.

Docker has limitations:

Security

Containers are isolated, but they're not equivalent to a separate physical machine.

Shared kernel

A container depends on the host kernel.

Persistent storage

Databases still need proper storage/backups.

Networking

Large deployments become complex.

Scaling

Running 500 containers manually isn't practical.

Orchestration

Eventually tools such as Kubernetes become relevant.

100. Where Kubernetes enters

Imagine one server:

```

Docker
├── backend
├── postgres
└── redis
  
```

Fine.

Now imagine:

```
100 servers
5000 containers
multiple regions
automatic scaling
health checks
rolling deployments
load balancing
```

You need orchestration.

That's where Kubernetes comes in.

```
Kubernetes
  ↓
Container runtime
  ↓
Containers
```

101. Your Docker learning hierarchy

I recommend learning in this order:

```
Linux basics
  ↓
Processes
  ↓
Filesystems
  ↓
Networking
  ↓
Permissions
  ↓
Docker fundamentals
  ↓
Images
  ↓
Containers
  ↓
Volumes
  ↓
Networking
  ↓
Dockerfile
  ↓
Docker Compose
  ↓
Docker Hub / Registries
  ↓
CI/CD
  ↓
Kubernetes
```

And you're already doing the right thing by learning Linux alongside Docker.

102. Linux concepts you should understand for Docker

You don't need to become a Linux kernel developer.

But know:

Processes

```
Bash
ps
top
htop
kill
```

Filesystem

```
Bash
ls
cd
pwd
cp
mv
rm
mkdir
find
```

Permissions

```
Bash
chmod
chown
```

Users

```
Bash
whoami
id
useradd
```

Networking

```
Bash
ip
ss
ping
curl
```

Processes/services

```
Bash
systemctl
journalctl
```

Environment

```
Bash
env
export
```

These concepts make Docker much easier.

103. The commands I'd actually memorize first

Don't try memorizing 100 commands.

Start with these:

Bash

```
docker version
docker info

docker pull
docker images

docker run
docker ps
docker ps -a

docker start
docker stop
docker restart
docker rm

docker exec
docker logs
docker inspect
docker stats

docker build
docker tag

docker network ls
docker network create
docker network inspect

docker volume ls
docker volume create
docker volume inspect

docker compose up
docker compose down
docker compose ps
docker compose logs
docker compose exec

docker system df
```

Those will cover a huge percentage of everyday Docker work.

104. Flags you should remember

These are extremely common:

Bash

```
-d
```

Detached/background.

Bash

```
-it
```

Interactive terminal.

```
Bash
--name
```

Container name.

```
Bash
-p
```

Publish port.

```
Bash
-v
```

Volume/bind mount.

```
Bash
-e
```

Environment variable.

```
Bash
--rm
```

Automatically remove container after exit.

```
Bash
--network
```

Choose Docker network.

```
Bash
--restart
```

Configure restart behavior.

```
Bash
--memory
```

Memory limit.

```
Bash
--cpus
```

CPU limit.

105. One command that combines everything

A realistic example:

Bash

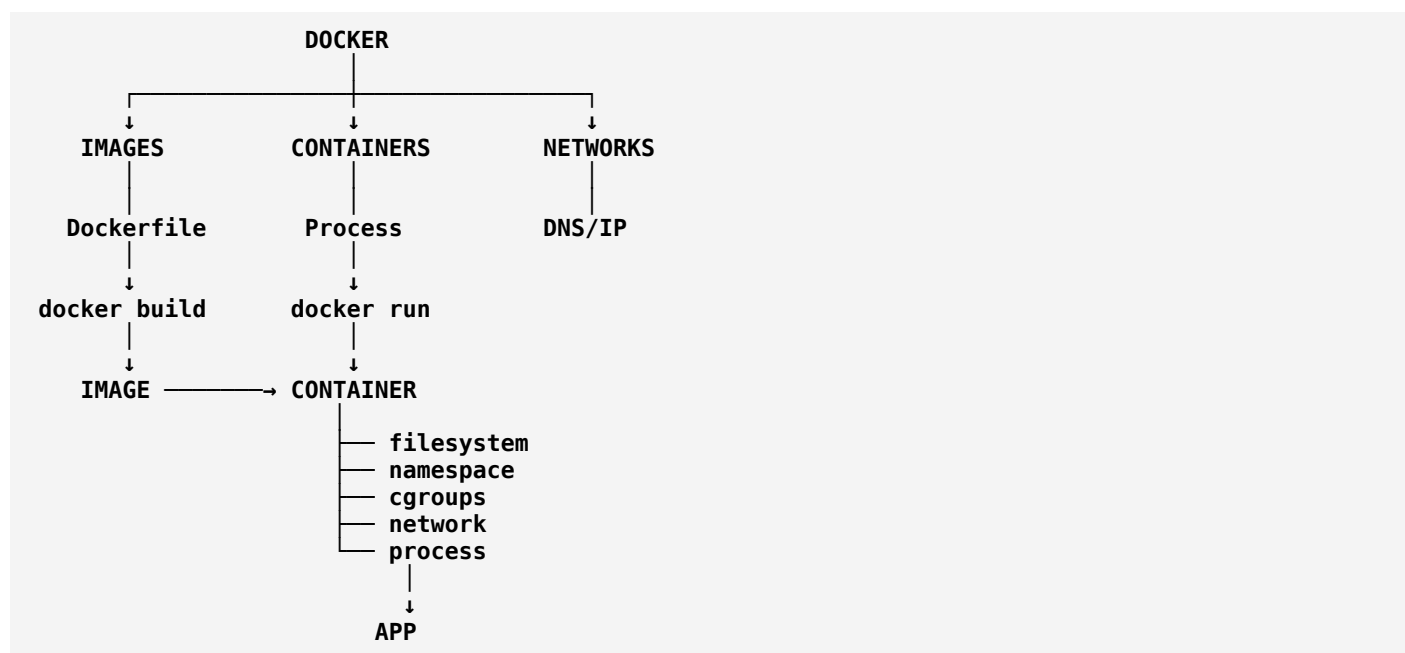
```
docker run -d \
  --name myapp \
  -p 8080:8000 \
  -e ENV=production \
  --restart unless-stopped \
  myapp:1.0
```

Read it as:

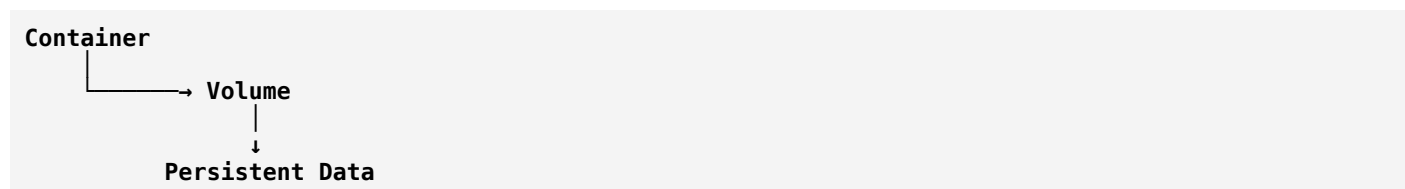
```
Run myapp:1.0
  ↓
in background
  ↓
name = myapp
  ↓
host 8080 → container 8000
  ↓
ENV=production
  ↓
restart if appropriate
```

106. The most important mental model

If you remember one diagram from this entire explanation, remember this:

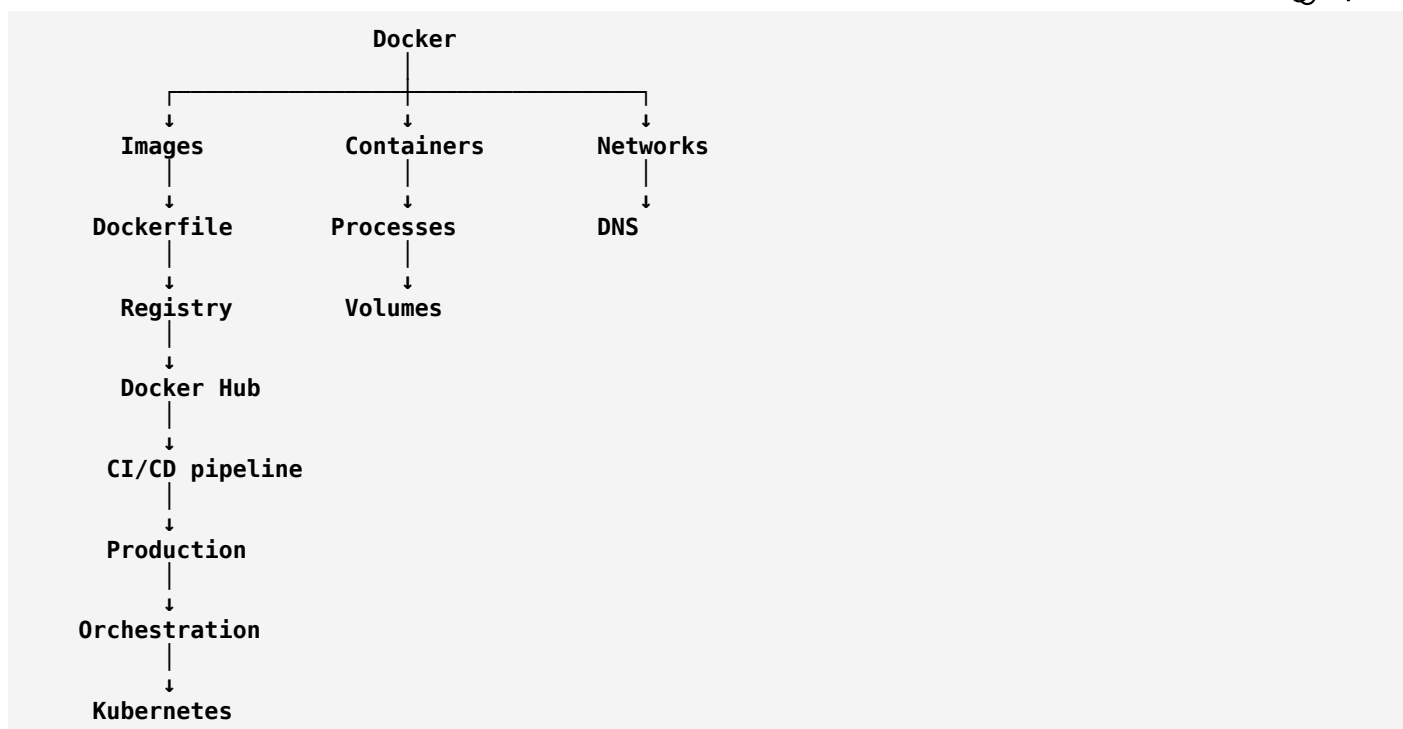


And for persistent data:



107. The complete Docker ecosystem

Eventually you'll encounter:



That is the bigger picture.

108. What you should NOT memorize yet

Don't waste time memorizing:

- Docker's internal source code
- containerd internals
- runc implementation details
- every namespace type
- every Docker networking driver
- every Compose option
- every Kubernetes object
- obscure Docker flags

Understand the **concepts** first.

Then commands become much easier.

109. What you should be able to explain after learning this

You should eventually be able to answer these without hesitation:

"What is a container?"

An isolated process/environment created using OS-level isolation mechanisms, packaged with an application and its dependencies.

"What is an image?"

A read-only, layered template from which containers are created.

"Image vs container?"

Image is the template; container is the runtime instance.

"Why Docker?"

To package, distribute and run applications consistently and efficiently.

"Docker vs VM?"

VMs virtualize complete machines/OS environments; containers isolate processes while sharing the host kernel.

"What are namespaces?"

Kernel mechanisms providing isolation of resources such as processes, networking and mounts.

"What are cgroups?"

Kernel mechanisms for controlling and accounting for resource usage.

"Why volumes?"

Containers are disposable, so persistent data should live outside the container's ephemeral writable layer.

"Why Dockerfile?"

To reproducibly build an image.

"Why Compose?"

To define and manage multi-container applications.

"Why Kubernetes?"

To orchestrate containers across infrastructure at scale.

110. Your Docker learning roadmap from here

Since you've already started running Ubuntu containers and are learning Linux, I'd do this:

Phase 1 — Linux fundamentals

Learn properly:

```
filesystem
processes
permissions
users
networking
environment variables
pipes
redirection
signals
services
```

Phase 2 — Docker fundamentals

Practice:

```
docker pull
docker run
docker ps
docker exec
docker logs
docker stop
docker rm
docker images
docker rmi
```

Phase 3 — Storage

Learn:

```
container filesystem
volumes
bind mounts
tmpfs
```

Phase 4 — Networking

Learn:

```
bridge
ports
DNS
container-to-container communication
custom networks
```

Phase 5 — Building images

Learn:

```
Dockerfile
FROM
RUN
COPY
WORKDIR
CMD
ENTRYPOINT
ENV
ARG
USER
EXPOSE
.dockerignore
```

Phase 6 — Compose

Build:

```
Django
+
PostgreSQL
+
Redis
```

with:

```
docker compose up
```

Phase 7 — Production concepts

Learn:

```
registries
image tagging
health checks
resource limits
logging
security
secrets
CI/CD
```

Phase 8 — Kubernetes

Only after the above:

```
Pod
Deployment
Service
ConfigMap
Secret
Volume
Ingress
Namespace
```

Then Kubernetes will actually make sense instead of looking like 47 YAML files written by an angry bureaucrat.

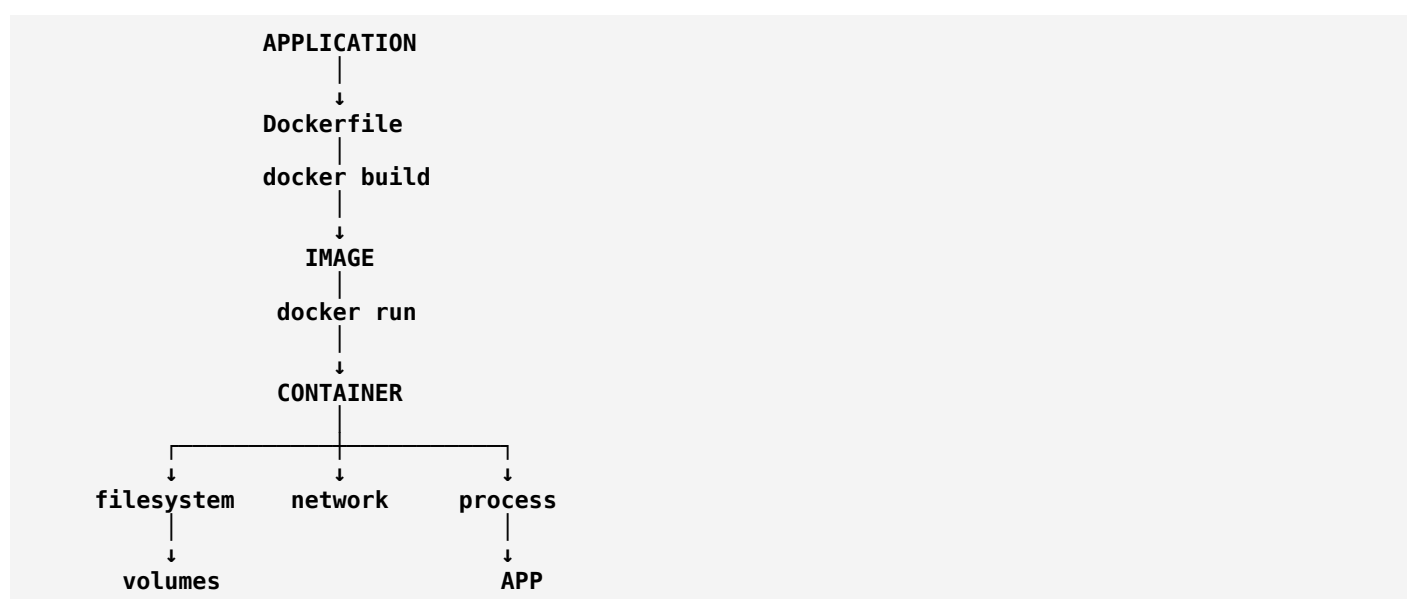
The 15 things I want you to remember

If this entire answer feels enormous, reduce it to these:

1. **Container $\neq$ VM.**
2. Containers normally share the host Linux kernel.
3. **Image = template.**
4. **Container = instance of an image.**
5. **Process = actual program running inside the container.**
6. `docker build` $\rightarrow$ Dockerfile $\rightarrow$ image.

7. `docker run` → image → container + process.
8. `docker ps` → running containers.
9. `docker ps -a` → all containers.
10. `docker exec -it` → execute a new process inside a running container.
11. `docker logs` → see application output.
12. `-p HOST:CONTAINER` → publish a container port.
13. **Volumes = persistent data.**
14. **Docker Compose = multiple containers/services together.**
15. **Kubernetes = orchestration at larger scale.**

And the deepest mental model is:



Once this picture is solid in your head, **Docker stops being a collection of commands and becomes a system you can reason about.** That's the point where learning Docker gets much easier.

Yesterday 2:43 PM



<https://youtu.be/rjjES5IsPdG?si=13DGh1pxEiRpSS6h> do I need to watch this tutorial after reading all what you have given in this chat

Yes — **but not because you need another explanation of Docker theory.** You should watch it for the **hands-on implementation.**

I checked the video you sent. It is the freeCodeCamp **“Learn Docker – Full DevOps Course for Deploying Containerized Apps”** course. Researchia+1

For you specifically

After the explanation I gave you, I would **not** recommend watching that tutorial passively from beginning to end.

Instead:

1. Read what I gave you → understand the concepts.

You now have the mental model:

```
Dockerfile
  ↓ docker build
Image
  ↓ docker run
Container
  ↓
Process
```

and:

```
Container
├── filesystem
├── network
├── namespaces
├── cgroups
└── application process
```

2. Now use the video as a practical lab.

When the instructor demonstrates something, **stop the video and do it yourself.**

For example:

```
Bash
docker pull nginx
docker images

docker run -d --name nginx1 -p 8080:80 nginx
docker ps

docker logs nginx1
docker exec -it nginx1 bash

docker stop nginx1
docker start nginx1
docker rm nginx1
```

Don't just watch him type those commands. **You type them.**

What I particularly want you to learn from that video

Focus heavily on:

- `docker run`
- containers and lifecycle
- images
- Dockerfile
- `docker build`
- ports
- volumes
- bind mounts

- Docker networking
- environment variables
- Docker Compose
- multi-container applications
- image registries
- containerizing an actual application

The course is explicitly aimed at deploying containerized applications, so this practical side is exactly what complements the conceptual explanation I gave you. 

Don't worry if he explains theory you've already read

If he spends 20 minutes explaining:

"What is Docker?"

and you already understand it, **don't feel obligated to take notes again.**

Watch it at 1.25–1.5× and pay attention when he actually starts doing things.

Your learning should become:

```
My explanation
↓
"I understand WHY"
↓
Video/tutorial
↓
"I see HOW"
↓
I reproduce it myself
↓
"I can actually DO it"
```

That's much better than watching three Docker tutorials and remembering three different definitions of a container. 😊

One important thing for your current Spring Boot learning

Since you're building a Spring Boot application with PostgreSQL, **don't stop at generic Docker examples.**

After this tutorial, your real milestone should be:

```
Spring Boot application
+
PostgreSQL
↓
Docker Compose
↓
Spring Boot container
+
PostgreSQL container
↓
persistent PostgreSQL volume
↓
Docker network
```

If you can build that yourself **without copying a tutorial**, I'll consider you to have actually learned the Docker fundamentals.

So my answer is:

Yes, watch the tutorial — but use it as a practical companion, not as another theory course. You've already done the conceptual groundwork. Now it's time to get your hands dirty with containers.



Are any thing I have to learn after this in docker ?? What you have given

Yes. What I gave you covers the **core Docker foundation**, but it is not literally everything you need to know for professional Docker usage.

Think of Docker knowledge in levels:

LEVEL 1 – Fundamentals	← You have this from my explanation
LEVEL 2 – Practical Docker	← Do the tutorial + build projects
LEVEL 3 – Production Docker	← Learn after that
LEVEL 4 – Kubernetes	← Later

What is still worth learning after my notes?

1. Dockerfile mastery ★★★★★

You know the basic instructions, but eventually learn:

- multi-stage builds
- build cache
- **.dockerignore**
- **ARG** vs **ENV**
- **CMD** vs **ENTRYPOINT**
- **COPY** vs **ADD**
- running as non-root
- image size optimization
- layer optimization
- build secrets
- BuildKit/buildx

For example, eventually you should understand why this:

```
dockerfile
FROM maven AS build
...
FROM eclipse-temurin
...
COPY --from=build ...
```

is useful.

This is particularly relevant for your **Spring Boot application**.

2. Docker Compose ★★★★★

Don't merely learn:

Bash

```
docker compose up
docker compose down
```

Learn how to actually design a stack:

Spring Boot



and understand:

- services
- networks
- volumes
- environment variables
- **depends_on**
- healthchecks
- restart policies
- **.env**
- profiles
- Compose overrides
- service-to-service DNS

This should be your next major target.

3. Docker networking ★★★★★

Go deeper into:

```
bridge networks
DNS
port publishing
container-to-container communication
host networking
network isolation
```

You should be able to explain why:

Spring Boot → localhost:5432

is **wrong** when PostgreSQL is another container.

Instead:

Spring Boot → postgres:5432

because **postgres** is the service/container name on the Docker network.

This will directly connect with the PostgreSQL work you've already been doing.

4. Storage ★★★★★

Understand properly:

```

container writable layer
  ↓
bind mount
  ↓
named volume
  ↓
tmpfs

```

And especially:

What happens to my data when I delete/recreate a container?

You should be completely comfortable with that question before running databases in Docker.

5. Docker security ★★★★★

This is often skipped by beginner tutorials.

Learn:

- why containers aren't security boundaries equivalent to VMs
- root vs non-root containers
- Linux capabilities
- seccomp
- read-only filesystem
- secrets
- image vulnerabilities
- trusted/minimal base images
- avoiding secrets in Dockerfiles
- Docker socket security

You don't need to become a security engineer, but you should know the basics.

6. Docker image registries ★★★★★

Learn the workflow:

```
Dockerfile
↓
docker build
↓
image
↓
docker tag
↓
docker push
↓
registry
↓
docker pull
↓
server
```

Understand:

- Docker Hub
- private registries
- tags
- image digests
- versioning
- authentication

7. Docker debugging ★★★★★

This is actually more important than memorizing obscure commands.

When your container says:

```
Exited (1)
```

you should know where to start.

Your toolbox:

```
Bash
docker ps -a
docker logs <container>
docker inspect <container>
docker exec -it <container> sh
docker stats
docker top <container>
docker events
```

And understand:

```
application problem?
network problem?
volume problem?
permission problem?
environment variable?
```

```
port?
image?
healthcheck?
```

That's real Docker skill.

8. Health checks ★★★★★

Understand:

```
dockerfile
HEALTHCHECK ...
```

and Compose health checks.

Why?

Because:

```
Container is running
```

does **not** necessarily mean:

```
Application is healthy
```

A PostgreSQL container could technically be running while your application still can't successfully use it.

9. Logging and monitoring ★★★★★

Understand:

```
stdout
stderr
docker logs
```

and basic resource monitoring:

```
Bash
docker stats
```

Later you'll encounter:

```
Prometheus
Grafana
ELK/OpenSearch
Loki
```

But you don't need those yet.

10. CI/CD with Docker ★★★★★

Eventually learn:

```

Git push
↓
CI
↓
Tests
↓
docker build
↓
docker push
↓
Registry
↓
Deployment

```

For example, GitHub Actions + Docker.

This is where Docker becomes much more useful professionally.

And then: Kubernetes

Don't jump to Kubernetes yet.

Once you're comfortable doing:

```

Spring Boot
+
PostgreSQL
+
Redis

```

using Docker Compose, then Kubernetes becomes the next logical step.

Your progression should be:

```

Linux
↓
Docker fundamentals
↓
Dockerfile
↓
Volumes
↓
Networking
↓
Docker Compose
↓
Docker debugging
↓
Docker security
↓
Docker registries
↓
CI/CD
↓
Kubernetes

```

How much Docker do YOU actually need?

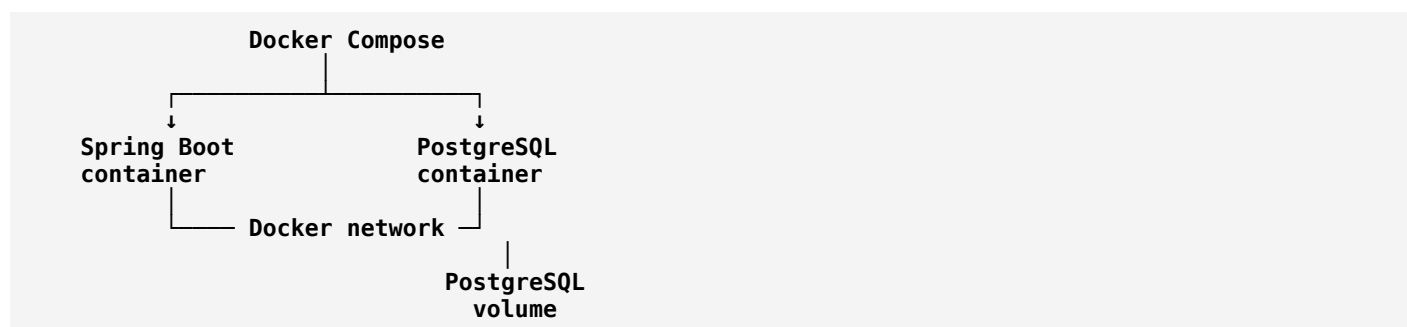
Given that you're learning **Spring Boot/backend development**, you don't need to become a Docker internals expert.

I'd divide it like this:

Topic	Priority
Containers/images	🔥🔥🔥🔥🔥
docker run, exec, logs, etc.	🔥🔥🔥🔥🔥
Dockerfile	🔥🔥🔥🔥🔥
Compose	🔥🔥🔥🔥🔥
Networking	🔥🔥🔥🔥🔥
Volumes	🔥🔥🔥🔥
Debugging	🔥🔥🔥🔥
Registries	🔥🔥🔥
Security	🔥🔥🔥
CI/CD	🔥🔥🔥🔥
Docker internals	🔥🔥
Kubernetes	🔥🔥🔥🔥 (later)

The actual stopping point

You can stop studying Docker theory when you can independently do this:



And you can:

1. Write the Dockerfile yourself.
2. Build the image.
3. Run it.
4. Publish its port.
5. Connect Spring Boot to PostgreSQL.
6. Persist PostgreSQL data with a volume.
7. Configure environment variables.
8. Create a Compose file.
9. Bring everything up/down.
10. Debug a container that fails.
11. Push your image to a registry.

At that point, stop collecting Docker tutorials. Build things.

That's the trap with Docker: you can spend three weeks learning `docker inspect` flags and still not know how to containerize your own application. Your goal should be **competence, not completion of the Docker encyclopedia.**